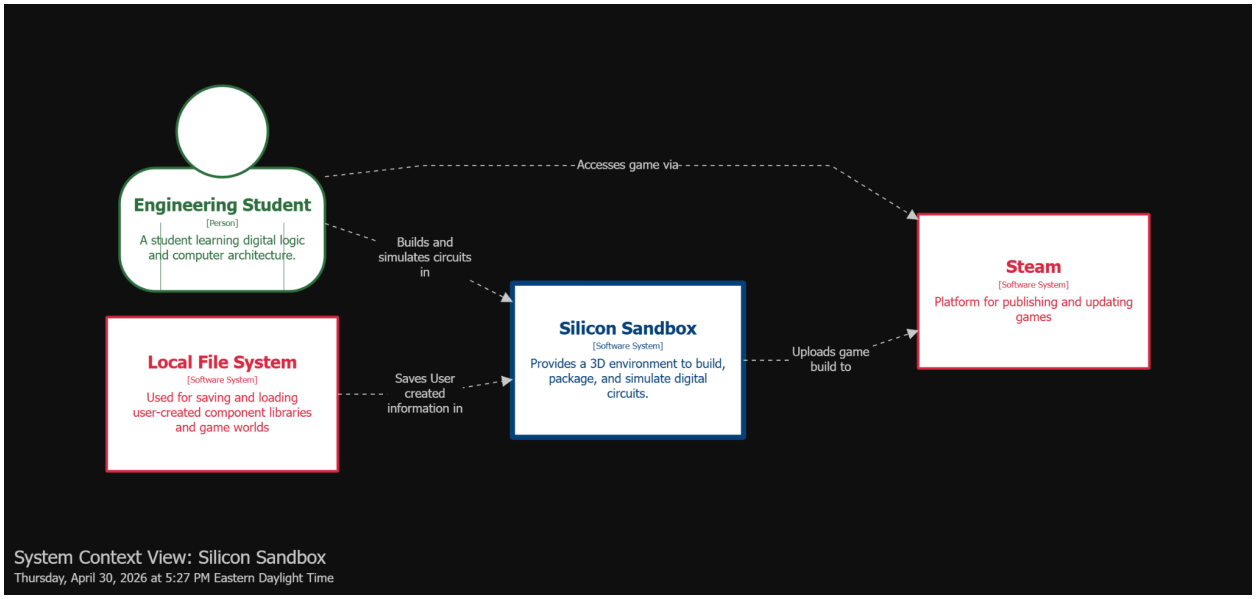
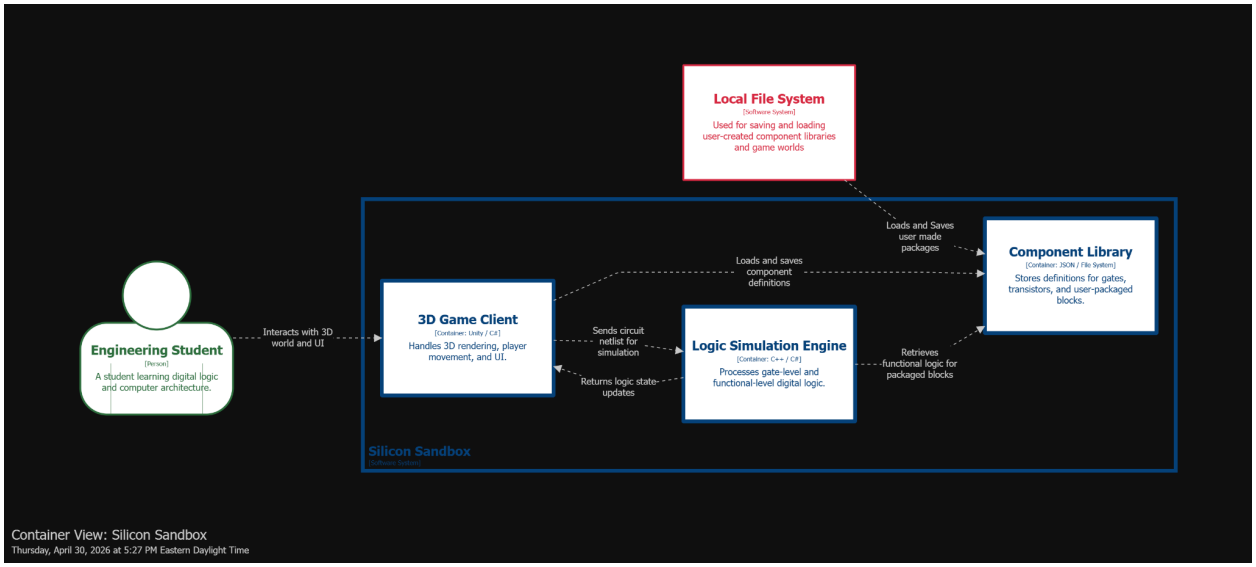
Design Diagrams

System Context Diagram



System Container Diagram



Technology Stack

Technology	System Scope	Reasoning
Unity	Core Game Engine	Manages the 3D environment, physics, and user interface
C#	Scripting and Logic	Language used in Unity for player information and interactions with the environment
C++	Logic Simulation	Language used for the logic simulation of the complex circuits
Native DLL Plugin	Logic Integration	A plugin used to bridge the C# code in Unity with the C++ code of the logic simulation
Newtonsoft.Json	Data Serialization	Used to effectively and efficiently save and load the data of the packages
NUnit	Automated Testing and Verification	Integrated into Unity to verify the correctness of game functionality logic gates and simulations
GitHub Actions	Automated Testing and Verification	Used to automate all testing of the product when the codebase is updated
GoogleTest	Automated Testing and Verification	Integrated into GitHub Actions to test the C++ logic simulation accuracy
Local File System	Storage	Using the local file system with the library to save local data and packages

Figma

https://www.figma.com/design/YQ1DUp8KyfUUQVUgs7JcfJ/Candidate-Designs?node-id=0-1&t=kAqW6KCatE7noVEE-1

Design Trade

Design Trade 1: Static Truth Table VS Hierarchical Simulation VS Stored circuit, Cached truth table (Hybrid)
These designs are in relation to the way packaged components are stored for future use.

1. Storing just a static truth table entails running through all possible inputs of a packaged circuit upon request to package. While running all possible inputs the outputs are stored into a truth table that can be referenced when the package is called.
2. Hierarchical Simulation stores the entire circuit component for component upon request to package. When a package is called upon the inputs are essentially rerouted though the stored components to recreate the original circuit.
3. The stored circuit & cached truth table method is a hybrid of both, where initially the circuit is stored in the same manner as in hierarchical simulation. As the package is called on in the future the inputs and outputs are stored in a truth table naturally, speeding up future calls by referencing the cached truth table.

Design	Logical Accuracy (*0.30)	Complexity (*0.25)	Scalability (*0.20)	Ease of Use (*0.15)	Expandability (*0.10)	Total
Static Truth Table	90	100	40	100	0	75
Hierarchical Simulation	100	20	100	100	100	80
Hybrid	95	70	50	100	100	81

The hybrid approach was chosen despite it not outperforming the other models in any sub category. Its total score outperforms because it's accommodating the weak points of each model with the strengths of the other. It performs as quickly during initial packaging as hierarchical simulation, but it is faster on average during long simulations. It also retains the original circuit even after a full truth table is built, so it is still expandable and editable later.

Design Trade 2: Event-Driven Simulator VS Cycle-Based Simulator VS Data-Parallel GPU Simulator

Design ID	Execution Speed (30%)	Memory Usage (20%)	Ease of Integration (15%)	Extensibility (20%)	Power Usage (15%)	Weighted Total Score
Event-Driven	9	6	7	9	10	8.25

Cycle-Based	7	10	9	7	6	7.75
Data-Parallel GPU	10	4	5	5	4	6.15

The event-driven simulator is highly efficient for large circuits as only the affected components are updated at once. It also easily supports complex timing propagation. It has a higher memory cost due to tracking events per gate. It is also architecturally complex and difficult to implement, however it will stay accurate and it is what most industry standard companies use.

Design Trade 3: Netlist synthesization

- DA-1: Direct Gate Copy and Store
    - Description: This approach will simply store the gates contained in the packaging bounding box in the exact layout that they are in. The linking tool will be sent the gates' positions as if they were still in the viewable game world, similar to how a pointer functions in software.
- DA-2: Brute Force Look Up Table
    - Description: For "n" number of inputs into the block, this approach will calculate all 2^n possible outputs associated with the possible inputs. A look up table will be produced, which can also be simplified into a K-map.
- DA-3: Traditional Logic synthesis
    - Description: This approach would mimic industry standard logic synthesis performed by modern EDA tools on hardware description languages like SystemVerilog.

	DA-1: Direct Gate Copy and Store	DA-2: Brute Force Look Up Table	DA-3: Traditional Logic synthesis
Time to Implement	10	8	1
Time to synthesize	9	2	4
Time to use during simulation	1	9	9
Creation of K-map	1	10	6
Creation of netlist	10	1	10
Point Totals	31	30	30

DA-1 may be the least efficient approach for use during simulation, but it is very easy to implement and is able to effectively store the packaged logic gates for use in re-expanding them when the user wants to study the lower levels of abstraction involved in their design. The ease of implementing such a design, and ability to recreate the logic gates make this the most optimal design choice.

GenAI Usage Statement

No AI was used on this assignment.